

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5

Total Marks:50

Annexure A

SIMULATION TASK

Code of Conduct:

*I M.V.Rakesh Reddy/192524164(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:M.V.Rakesh Reddy

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model

		independently			
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Annexure A

SIMULATION TASK

QUESTIONS

1. Given the three-address code sequence:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

Simulate the **peephole optimization** process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied, including algebraic simplification, constant folding, and dead code elimination.

2. Partition the following intermediate code into **basic blocks** and construct the corresponding **control flow graph**. Identify all leader statements and mention the rules used to identify them:

```
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2
```

3. Construct the **DAG** for the following basic block:

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

Using the DAG, identify all **common sub-expressions**. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

4. For a target machine with two registers **R0** and **R1**, simulate the working of a **simple code generator** for the code:

```
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
```

Show the **register descriptor** and **address descriptor** after each instruction is processed.

5. For a **RISC-style target machine** with instructions **LOAD**, **STORE**, **ADD**, **SUB**, **MUL**, **MOV**, simulate target code generation for the expression:

$(a - b) * (c + d) - e$

Show the complete sequence of target instructions generated. Also explain the **runtime storage management** and **stack frame layout** used during the computation.

Given the three-address code sequence:

```
t1 = a + b,
```

```

t2 = t1 + 0,
t3 = t2 * 1,
if t3 == 0 goto L1,
t4 = t3 + t3

```

RUBRICS FOR CALCULATION

Simulation tools					
Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – ANSWERS

COURSE CODE: CSA14 COURSE NAME: COMPILER DESIGN

COURSE OUTCOME: CO5 – Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code.

Simulation Task – Solved Answers

Student: M.V. RAKESH REDDY Reg. No.: 192524164 B.Tech AI & DS

Question 1

Given three-address code:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

Peephole optimization steps

Step	Optimization	Result
1	Algebraic simplification: $x \times 1 = x$	$t1 = x$
2	Substitute t1 into t2	$t2 = x + y$
3	Algebraic simplification: $A - 0 = A$	$t3 = x + y$
4	Constant folding: $A \times 0 = 0$	$t4 = 0$
5	Constant condition: $0 \neq 0$ is false	Conditional jump is removed
6	Dead code elimination	$t5 = y + y$ remains because it is reachable and is not proven dead from the given fra

Optimized instruction sequence:

```
t1 = x
t2 = x + y
t3 = t2
t4 = 0
t5 = y + y
```

A more compact final form after removing redundant temporaries is: **$t2 = x + y$; $t5 = y + y$** . The jump is eliminated because t4 is always zero.

Question 2

Intermediate code and line numbers are taken directly from the supplied assessment.

Line	Statement
1	$x = a + b$
2	$y = x * c$
3	if $y > 50$ goto 6
4	$z = y - d$
5	goto 7
6	$z = y + d$
7	$w = z * 2$

Leader identification rules

1. The first statement of the code is always a leader. 2. Any statement that is the target of a conditional or unconditional jump is a leader. 3. Any statement immediately following a jump statement is a leader.

Leaders

Line 1 – first statement.

Line 4 – statement immediately following the conditional jump at line 3.

Line 6 – target of the conditional jump at line 3.

Line 7 – target of the unconditional jump at line 5.

Basic blocks

Block	Statements
B1	(1) $x = a + b$ (2) $y = x * c$ (3) if $y > 50$ goto 6
B2	(4) $z = y - d$ (5) goto 7
B3	(6) $z = y + d$
B4	(7) $w = z * 2$

Control Flow Graph

B1 → B2 (false branch of $y > 50$)

B1 → B3 (true branch to line 6)

B2 → B4 (goto 7)

B3 → B4 (normal flow)

CFG representation: **B1** false **B2**  **B4**

B1 true **B3**  **B4**

Question 3

Basic block:

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

DAG construction

Create leaf nodes for a and b. The expression $a + b$ is computed once as a common internal node N1. Variables p, q and s are attached to N1. Then create $N2 = p \times q$ (equivalently $N1 \times N1$), and $N3 = r + s$.

DAG node	Expression	Attached names
N1	$a + b$	p, q, s
N2	$N1 * N1$	r
N3	$N2 + N1$	t

Common sub-expression

The expression $a + b$ occurs three times: in p, q and s. The DAG represents it using one node, so the addition is performed only once.

Optimized target code

```
MOV R0, a
ADD R0, b      ; R0 = a + b
MOV R1, R0     ; preserve value for second operand
MUL R1, R0     ; R1 = (a + b)2
ADD R1, R0     ; R1 = (a + b)2 + (a + b)
MOV t, R1
```

Register decision: R0 holds the shared value $a + b$ because it is needed by p, q and s. R1 is used for the multiplication and final addition. This avoids recomputing $a + b$.

Question 4

Target machine: two registers R0 and R1.

Three-address code:

```
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
```

Register descriptor and address descriptor

After instruction	R0	R1	Address descriptor (main values)
t1 = a+b	t1	free	t1: R0; a,b: memory
t2 = c*d	t1	t2	t1: R0; t2: R1; c,d: memory
t3 = t1-t2	t3	t2	t3: R0; t2: R1; t1 value consumed
t4 = e+f	t3	t4	t3: R0; t4: R1; e,f: memory
t5 = t3/t4	t5	free	t5: R0; t3,t4 values consumed

The descriptors show where each current value is located. When a temporary is no longer needed, its register can be reused. With only two registers, the generator keeps t1 while computing t2, then replaces the consumed values as the computation proceeds.

Question 5

Expression: $(a - b) \times (c + d) - e$

RISC-style target instructions using LOAD, STORE, ADD, SUB, MUL and MOV:

```
LOAD R0, a
LOAD R1, b
SUB R0, R1      ; R0 = a - b
STORE TEMP1, R0
LOAD R0, c
LOAD R1, d
ADD R0, R1      ; R0 = c + d
STORE TEMP2, R0
LOAD R0, TEMP1
LOAD R1, TEMP2
MUL R0, R1      ; R0 = (a-b)(c+d)
LOAD R1, e
SUB R0, R1      ; R0 = (a-b)(c+d)-e
STORE RESULT, R0
```

Register usage

R0 is the primary accumulator. R1 is used as the second operand. TEMP1 stores the first sub-expression because the two-register target cannot keep all intermediate values simultaneously. TEMP2 stores the second sub-expression. After multiplication, e is loaded into R1 and subtracted from R0.

Runtime storage management and stack frame

At run time, the stack frame can contain local temporaries such as TEMP1 and TEMP2, plus the final RESULT when required. A simple layout is: higher address → saved return/control information → local temporaries TEMP1/TEMP2 → spill space → lower address. The exact physical layout depends on the target calling convention; the assessment only specifies the available RISC instructions, so this conceptual layout is used.

Temporary values are stored to memory when registers are insufficient and loaded again when needed. Values that are dead after an operation can have their registers reused, reducing memory traffic and improving generated code.

Additional optimization example from the assessment

Given: $t1 = a + b$; $t2 = t1 + 0$; $t3 = t2 * 1$; if $t3 == 0$ goto L1; $t4 = t3 + t3$.

Algebraic simplification gives $t2 = t1$ and $t3 = t2$. Therefore the condition becomes **if $t1 == 0$ goto L1**, and the final statement becomes **$t4 = t1 + t1$** . This removes the redundant +0 and ×1 operations.

Conclusion

Peephole optimization removes redundant operations, basic-block formation identifies straight-line regions and control-flow edges, DAG representation removes repeated common sub-expressions, and register/address descriptors guide efficient target-code generation. Together these techniques produce shorter and more efficient target code.